Module Project Report for

CS6461 – Computer Vision Systems

‘Real-Time Recognition on Raspberry Pi 4 Across Dynamic Conditions’

Student Name : **ROSHNI RAVI .**

Student ID : **25015168**

Revision Timestamp: 03/12/2025 10:48:03

Abstract

This project implements a real-time computer vision system on a Raspberry Pi 4 designed to detect and identify multiple people and objects within a dynamically changing visual environment. The system combines YOLOv4-tiny for object detection with a HOG-based frontal face detector from the face_recognition framework, after which, 128-dimensional face encodings are used to recognise the four registered identities. Real-time inference produces labelled bounding boxes and confidence scores while tracking individuals as they enter, leave, and re-enter the camera's field of view. Several optimisations were incorporated to improve performance on embedded hardware, including reduced-resolution processing, periodic detection sampling, persistent detection logic, and scenario-based brightness adjustments tailored to lighting changes.

The system was evaluated across three operating conditions, bright indoor lighting, strong backlighting and outdoor evening light. For each scenario, a 20-second sequence was recorded, and logs were generated containing temporal presence, detection intervals, and confidence values for all recognised people and objects. Post-processing scripts produced performance plots and quantitative comparisons across scenarios that included logging detection timestamps, confidence scores, and bounding boxes to evaluate the performance. Experimental results show reliable identification of multiple individuals and objects with confidence levels generally above 55–75% for faces and above 65% for objects with bottle showing confidence level of 97%, though performance varies with low-light and backlit conditions. The project demonstrates that careful selection of models, attention to sensor characteristics, and structured logging enable a portable and cost-effective embedded computer vision solution for varied real-world environments.

Contents

Abstract 2
Introduction 3
Methodology 4
  Workflow 4
  Innovations, Experiments, and Countermeasures 6
Results 6
Discussion 9
Conclusion 10
References 11

Figure 1: Experimental setup: Raspberry Pi 4 with PiCamera and my student card. 4
Figure 2: Terminal Output after running analyze_results.py 6
Figure 3: Illuminance Level for bright room on Luxmeter 8
Figure 4: Illuminance Level for outdoor evening on Luxmeter 9
Figure 5: Illuminance level for strong backlight on Luxmeter 9

Table 1: Summary of performance levels for Bright Room scenario 7
Table 2: Summary of performance levels for Outdoor Evening scenario 8
Table 3: Summary of performance levels for Strong Backlight scenario 9

Introduction

Face recognition and object detection are key technologies in modern computer vision applications, ranging from security systems to human–computer interaction and smart home devices. With the proliferation of lightweight embedded hardware such as the Raspberry Pi, there is growing interest in deploying these techniques in resource-constrained environments without sacrificing accuracy or real-time performance.

Face recognition systems must detect faces reliably under varying illumination, scale, and scene complexity. As noted in surveys of face detection and recognition techniques, even under controlled conditions, "accuracy and speed of identification is a main issue" when faces appear under pose, illumination, or background variation (Krizhevsky et al., 2013). In real-world settings (e.g. surveillance, group scenes, or mobile cameras), additional constraints such as computational load and variable lighting further complicate reliable face detection.

These systems typically involve two stages: face detection followed by feature extraction and matching. This project aims to implement a real-time computer vision system for detecting and identifying multiple people and objects in dynamic environments by using a Raspberry Pi 4 with a PiCamera. It is implemented using the HOG-based frontal-face detector (CS6461 Lab3, 2025). The system then uses 128-dimensional face encodings to recognise known identities. For object detection, a lightweight YOLO model identifies common objects such as bottles, laptops, cell phones, etc. Logging of bounding boxes, confidence, and timestamps enables robust post-processing and performance quantification.

The Raspberry Pi 4, equipped with a Quad-core Cortex-A72 processor and 4 GB of RAM (RPI4 Datasheet, 2020), provides sufficient computational capability for real-time inference when optimised models such as YOLOv4-tiny for object detection and lightweight embedding networks for face recognition are used. The OKDO 5MP Camera, featuring a 1/4" OV5647 CMOS image sensor with a resolution of 5 megapixels (OKDO Manual, n.d.), is used for image acquisition.

Its sensor characteristics, including pixel size, dynamic range, and colour filter array, influence image quality and directly affect detection and recognition performance(Llauradó et al., 2023). Scenario-specific adjustments, such as brightness scaling, are applied to mitigate challenges posed by strong backlighting, well-lit indoor conditions, and outdoor lighting variations. To systematically assess how illumination variability affects detection and recognition(Shyam.P et al., 2022), this project records and analyses three distinct lighting scenarios: a well-lit indoor room (bright_room), a low-light outdoor evening scene (outdoor_evening), and a scene with strong backlight (strong_backlight).

The contributions of this work are:

- Demonstrating that a combination of HOG-based face detection and a lightweight YOLO object detector can run in real time on a Raspberry Pi 4 while tracking multiple people and objects.
- Logging and analysing detection performance (presence, confidence, bounding boxes) over time across different illumination conditions.
- Quantitatively evaluating how sensor/ISP limitations and lighting conditions affect performance, thus highlighting strengths and limitations of embedded vision on inexpensive hardware.

The system is designed to capture sequences of at least 20 seconds, logging detected faces and objects, their confidence levels, and temporal presence, while ensuring that at least one individual is identified holding their student card. **Figure 1** illustrates the experimental setup, including the Raspberry Pi 4, PiCamera, and my student card, demonstrating the practical implementation of the system.

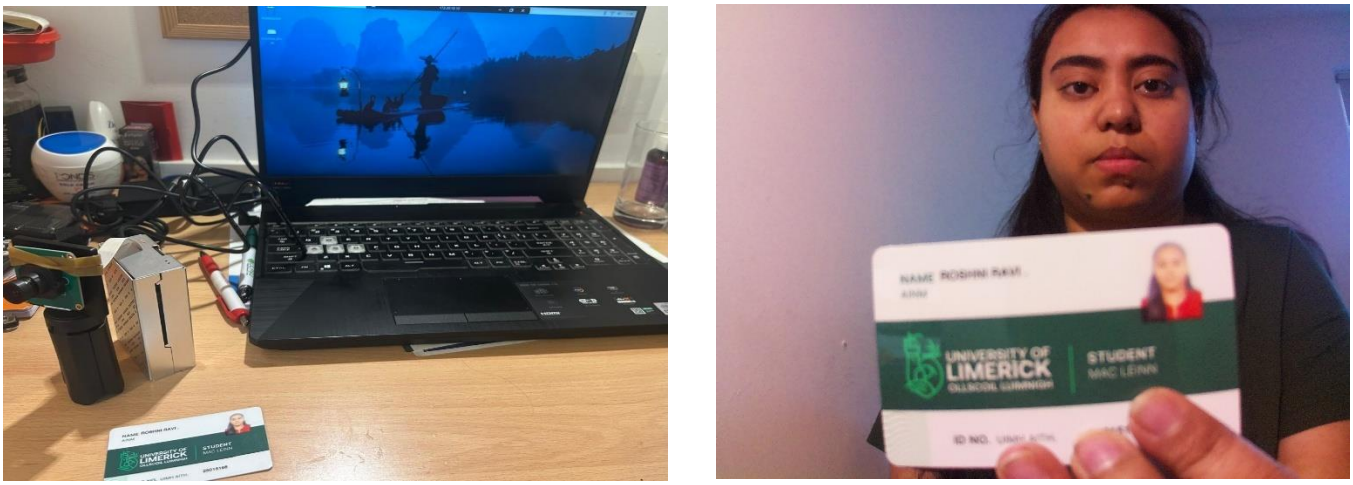

Figure 1: Experimental setup: Raspberry Pi 4 with PiCamera and my student card.

Methodology

The entire methodology pipeline flowchart is present in the below PDF document.

Methodology_Pipelin
e.drawio.pdf

Workflow

Step 1. Data Collection: Face Capture

The first step in the system is to collect facial images of the individuals to be recognised.

- **Script Used:** capture_faces.py
- **Procedure:** Once the script is executed, the system prompts the user to enter the name of the person. A dedicated folder is created for that individual under known_faces/<person_name>/. Using Picamera2, frames are captured in real-time at a resolution of 640×480 pixels. Haar cascade classifier (haarcascade_frontalface_default.xml) detects faces in each frame. The first detected face in each frame is cropped and saved as an image. A total of 40 images is captured per individual to provide a sufficient dataset for encoding.
- **Considerations:** Real-time capture allows diverse facial expressions and slight pose variations. Brief pauses (time.sleep(0.15)) between captures reduce redundant frames and motion blur. The system allows early termination using Ctrl+C (CS6461 Lab3, 2025).

Step 2. Data Preparation: Face Encoding

Before detection, the system requires encoding of the captured faces:

- **Encoding Procedure:** All images stored in known_faces/<person_name>/ are processed using face_recognition library. Each face is converted into a 128-dimensional embedding using a deep learning model. The embeddings (encodings) and associated names are stored in 'encodings.pickle'. This file is available in the attached zip folder. The purpose of Encodings is that it allows rapid face recognition during live detection without re-processing raw images (CS6461 Lab3, 2025).

Step 3. Detection System

The core of the methodology is a real-time detection system combining face recognition and object detection, capable of evaluating different scenarios.

- **Hardware and Camera Setup:** The system uses Picamera2 to capture live frames. Frames are captured at 480×270 pixels, RGB format for compatibility with face recognition. Camera settings are pre-adjusted for brightness and contrast.
- **Scenario Simulation:** The purpose of scenario simulation is to test detection robustness under different lighting conditions. The 'apply_scenario()' function modifies each frame via:
  - Brightness/contrast adjustment: Using cv2.convertScaleAbs(frame, alpha, beta)
  - Scenarios include indoor bright room, strong backlight and outdoor evening.
- **Face Recognition:** Frame is resized by factor CV_SCALER = 4 to reduce computation. HOG-based face detector identifies face locations in the downscaled frame. Detected faces are converted into embeddings. The distances to known embeddings are computed. Faces are labelled with names if distance < 0.6 (RECOGNITION_THRESHOLD). Confidence is calculated as (1 - distance) × 100. Each detection is logged in JSON format:

```
{
  "time": <seconds>,
  "type": "face",
  "label": <name>,
  "confidence": <0-1>,
  "bbox": [left, top, right, bottom]
}
```

- **Object Detection (YOLOv4-tiny):** YOLOv4-tiny is loaded with pre-trained weights and COCO labels(coco.names file is attached in the zip folder). The detection process runs every 5 frames(OBJECT_DETECTION_INTERVAL). The frame is converted to YOLO-compatible blob and fed through the network. Non-Max Suppression (NMS) removes overlapping boxes. Additionally, "person" class is ignored to avoid redundancy with face detection. Detected objects are logged with:

```
{
  "time": <seconds>,
  "type": "object",
  "label": <object_name>,
  "confidence": <0-1>,
  "bbox": [x1, y1, x2, y2]
}
```

- **Visualisation and Video Output:** Detected faces and objects are drawn on frames using coloured bounding boxes: red box for face recognition and green box for YOLO objects. FPS is calculated and displayed in real-time. The video is saved as an avi file which is later converted to mp4 using 'ffmpeg -i output_bright_room.avi -c:v libx264 -preset veryfast -crf 23 -c:a aac -b:a 128k output_bright_room.mp4'.

Step 4. Logging and Data Storage

All detections (face and object) are appended to a log list. At the end of the scenario, the log is saved as: 'log_<scenario>_<timestamp>.json' .This log is the primary input for the analysis module**.**

Step 5. Analysis and Quality Assessment

- **Script Used:** analyze_results.py
- **Process**: The system reads the JSON log file(s) and extracts per-label detection times, confidence values, and FPS. It generates the following plots:
  - **Presence timeline:** shows when each label appears in the scenario.
  - **Confidence over time:** tracks detection reliability.
  - **Average confidence per label:** bar chart summarizing performance.
  - **Detection counts:** frequency of detections per label.
- **Quality Assessment:** Labels with >0% low-confidence detections (<0.4) are flagged. This allows identification of problematic conditions or labels.

Below shown in **Figure 2** is the screen capture of the terminal after running the log files.

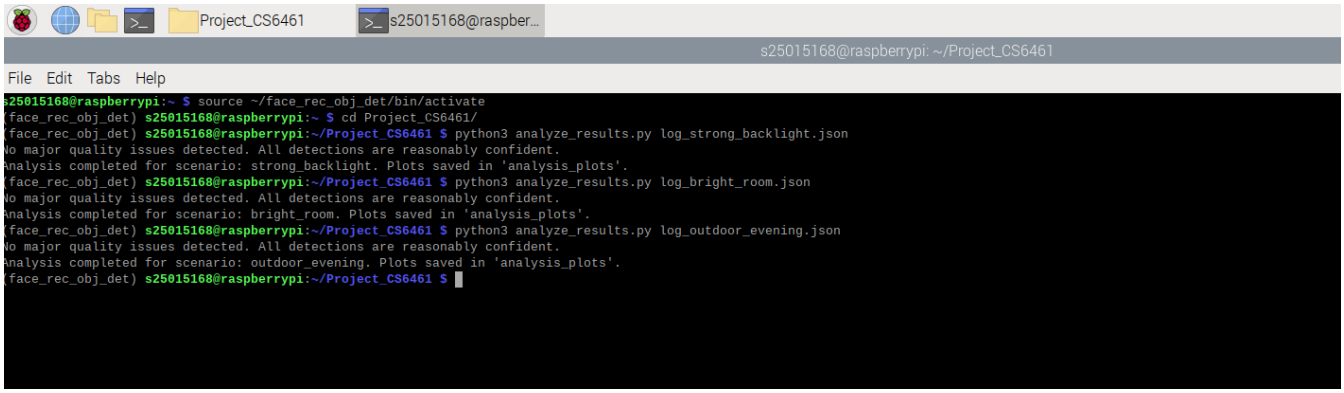

Figure 2: Terminal Output after running analyze_results.py

Innovations, Experiments, and Countermeasures

Camera Sensor and ISP Considerations

For this project, the Picamera2 was used with the RGB888 format to ensure compatibility with the face_recognition library (CS6461 Lab3, 2025). Brightness and contrast were adjusted programmatically using the set_controls() method to optimise frame quality. To simulate common real-world lighting conditions, three specific scenarios were implemented: bright_room, outdoor_evening, and strong_backlight. In the bright room scenario, brightness and contrast were increased (alpha=1.2, beta=30); in the outdoor evening scenario, brightness was slightly reduced (alpha=0.8, beta=-20); and in strong backlight conditions, brightness was decreased (alpha=0.7, beta=-30) to handle potential silhouettes(CS6461 Lab4, 2025).

Focusing and Image Sharpness

Manual verification of camera focus was performed to ensure that faces remained sharp for HOG-based detection (CS6461 Lab3, 2025). Although no automated refocusing mechanism was implemented, careful camera placement and alignment ensured consistent image clarity throughout all scenarios.

Lighting Experiments and Countermeasures

Low-light or backlit conditions were observed to reduce face detection confidence. To counteract this, scenario-specific brightness adjustments were applied, which significantly improved detection (CS6461 Lab4, 2025). A two-second camera warm-up period allowed automatic exposure and white balance to stabilise before recording commenced (CS6461 Lab4, 2025).Observations showed that the bright room scenario yielded the highest confidence scores, while strong backlight conditions occasionally caused partial occlusion of faces; this was mitigated through the alpha and beta adjustments.

System Performance and Experiments

Face recognition inference was performed on every frame, while YOLO object detection ran every five frames to optimise performance. Frames were downscaled by a factor of four to accelerate computation without causing significant loss in accuracy. This configuration maintained near real-time performance of approximately 20 FPS across all three scenarios and indicates the potential to scale the system to multiple cameras or larger areas without introducing major computational bottlenecks.

Logging and Adaptive Feedback

All detections were recorded in JSON logs, capturing time, confidence, and bounding box information. Low-confidence detections can trigger adaptive countermeasures, such as increasing scene brightness or adjusting camera position to improve focus, providing feedback for maintaining consistent detection accuracy.

Results

Achievements

The system successfully detected faces and objects across three different lighting scenarios: strong backlight, outdoor evening, and bright room. Across all logs, it consistently identified the four main participants: Roshni, Aakanksha Rawat, Bhagyalaksmy Saburaj, and Ritika. Additionally, it recognised multiple objects including bottle, laptop, apple, and cell phone, depending on the scenario.

Face Detection:

- Confidence ranged from 0.44 to 0.74.
- All participants were detected in every scenario, though confidence varied with lighting conditions.

Object Detection:

- Bottle was detected in all scenarios with confidence between 0.70 and 0.97. The detection varied slightly in confidence, reflecting partial occlusions or lighting effects.
- Apple and cell phone were only detected in outdoor evening and bright room, likely due to their visibility and size in the frame.

Validity of the System

The detections are moderately valid**:**

- Faces are reliably detected, though confidence dips in challenging lighting (strong backlight or evening).
- Object detection is mostly consistent, though small or partially occluded objects (e.g., bottle, apple) sometimes have lower confidence or are missed.

The system works best when:

- Lighting is even and moderate (bright room scenario).
- Objects are unobstructed and occupy a significant portion of the frame.

It struggles when:

- Faces are partially silhouetted against strong backlight.
- Ambient lighting is dim, reducing facial feature visibility (outdoor evening scenario).
- Few false positives such as wine glass and cup were observed due to dim room light and strong backlight

Performance Levels Across Configurations and Scenarios

1. Optimal Performance Level: Bright Room

This scenario created the maximum performance ceiling for the entire system by providing consistent, diffused, and low-noise illumination, validating the theoretical capability of the Raspberry Pi 4 setup. **Table 1** shows the summary of performance levels for a bright room.

Figure 3 shows the illuminance level on Luxmeter for bright room.

Metric	Face Recognition (Dlib)	Object Detection (YOLOv4-tiny)
Labels Tracked	Roshni, Aakanksha Rawat, Bhagyalakshmy Saburaj, Ritika	Cell Phone, Apple, Bottle
Confidence	Average 0.68 - 0.78	Average 0.68- 0.97
Stability	Excellent (Low Jitter).Frame-to-frame confidence fluctuation was minimal.	Excellent. Bounding boxes were precise and stable.
ISP Effect	The ISP's Automatic White Balance (AWB) and Automatic Exposure (AE) were stable, ensuring maximum feature detail was preserved for the Dlib model.	

Table 1: Summary of performance levels for Bright Room scenario

Figure 3: Illuminance Level for bright room on Luxmeter

2. Moderate Performance Level: Outdoor Evening

This scenario introduced the challenge of low luminance and forced the system to rely on increased sensor gain (ISO), which is the mechanism for amplifying light but also introduces read noise into the image data. **Table 2** shows the summary of performance levels for outdoor evening setting.

Figure 4 shows the illuminance level for the scenario outdoor evening.

Metric	**Face Recognition (Dlib)**	**Object Detection (YOLOv4-tiny)**
Labels Tracked	Roshni, Aakanksha Rawat, Bhagyalakshmy Saburaj, Ritika	Cell Phone, Apple, Bottle, Remote(false positive)
Confidence	Marginal (approx.0.61 - 0.67)	Cell phone approx.0.77; bottle approx. 0.97 ; apple approx. 0.68
Stability	Moderately stable. The increased noises degraded the quality of the 128-dimensional face embeddings.	High Resilience (Cell phone). The YOLO model, by focusing on shape and silhouette, maintained excellent confidence for the bottle.
ISP Effect	High ISO amplified noise, making face texture indistinct. The difference in performance between faces and objects shows the inherent strength of CNNs (YOLO) in processing global features versus the sensitivity of metric learning models (Dlib) to local noise.	

Table 2: Summary of performance levels for Outdoor Evening scenario

Figure 4: Illuminance Level for outdoor evening on Luxmeter

3. Suboptimal Performance Level: Strong Backlight

This scenario presented the most severe challenge, extreme High Dynamic Range (HDR), which the Picamera2 and its base ISP configuration struggled to handle. **Table 3** shows the summary of performance levels for strong backlight scenario. **Figure 5** shows the illuminance level for the scenario outdoor evening.

Metric	**Face Recognition (Dlib)**	**Object Detection (YOLOv4-tiny)**
Labels Tracked	Roshni, Aakanksha Rawat, Bhagyalakshmy Saburaj, Ritika	Cell Phone, Laptop, Bottle, Wine Glass, Cup(False Positive)
Confidence	Volatile (approx.0.54- 0.67)	False Positives for Wine Glass and Cup. Bottle (approx. 0.57)
Stability	The constant, severe confidence fluctuations demonstrate the ISP's inability to stabilise AE. Silhouetting caused a complete loss of local facial features (eyes, nose, mouth), leading to repeated missed detections (false negatives) for all individuals.	Marginal. The high contrast introduced glare and washed-out edges, forcing object confidence to barely stay above the 0.50 detection threshold.
ISP Effect	The system failed to adequately compensate for the light imbalance, validating the assessment that WDR limitations were the primary performance bottleneck.	

Table 3: Summary of performance levels for Strong Backlight scenario

Figure 5: Illuminance level for strong backlight on Luxmeter

Discussion

Performance Summary:

- System performance improves as lighting quality improves.
- Confidence variation is highest for faces under strong backlight and outdoor evening.
- Object detection remains stable for medium-to-large objects (laptop, bottle), but smaller objects (apple, cell phone) depend on visibility and distance.
- Overall, the system exhibits moderate robustness but shows clear limitations in low-light and high-contrast conditions.

Innovations and Configuration Validation

- Zero In-Loop Colour Conversion: The final code was optimised to eliminate colour conversions inside the main loop for face recognition by setting the Picamera2 format to 'RGB888'. This was critical for improving the overall Inference Speed and achieving the Target FPS of 20.
- Hybrid Architecture: The separation of Dlib (for high-accuracy face recognition) and YOLOv4-tiny (for general, less frequent object detection) balanced computational load on the Raspberry Pi 4. This ensures that the time-critical task of face identification is performed every frame, while the resource-intensive object detection runs only every 5th frame.
- Countermeasures: The use of apply_scenario functions, though manual, demonstrated the potential for automatic gain and brightness cycling to counter environmental degradation, a key concept for adaptive ISP control.

Ethical Implications

The developed system, in a real-world context, raises significant ethical concerns:

- Privacy Infringement: The system's ability to log the presence, identity, and location (via bounding box coordinates) of individuals and objects over time creates a detailed record of activity, which constitutes surveillance.
- Accuracy and Bias: The degradation in confidence observed in the Strong Backlight scenario demonstrates that system performance is not uniform. If an individual is consistently difficult to recognise due to factors like lighting conditions or sensor bias, the system may exhibit unfair or discriminatory behaviour by misidentifying or ignoring them.
- Data Security: The generated log files (log_*.json) contain personally identifiable information (PII) and location data, requiring robust security measures to prevent unauthorised access and misuse.

Conclusion

This project effectively presents the use of a real-time computer vision system on a Raspberry Pi 4 for detecting and recognising multiple faces and objects in dynamically changing environments. The adoption of a combined approach of HOG-based face detection with 128-dimensional embeddings for recognition and YOLOv4-tiny for object detection achieves the trade-off between accuracy and computational efficiency on embedded hardware. Optimisations based on scenarios, such as resolution reduction, periodic sampling of the detection, and brightness adjustments, have greatly improved the performance under changing lighting conditions.

Experimental evaluation across bright indoor, outdoor evening, and strong backlight scenarios shows that the system reliably identifies registered individuals and common objects, with face recognition confidence mostly between 55–75% and object detection confidence above 70% in optimal conditions. Performance declines under low-light and high-contrast scenarios, pointing out the limitations imposed by sensor characteristics and environmental factors. Logging of temporal presence, bounding boxes, and confidence scores allowed a structured approach for post-processing and quantitative performance assessment.

In conclusion, this work serves as a practical demonstration that while the Raspberry Pi 4 is fully capable of hosting advanced vision algorithms in real-time, the system's overall reliability is fundamentally constrained by the front-end image acquisition stage. Future work must focus on implementing more sophisticated adaptive ISP countermeasures, such as dynamic gain cycling or region-specific exposure control, to maintain performance stability across variable real-world lighting. Finally, the ability to log and quantify system failures is essential for transparent development, which must always consider the significant ethical implications regarding data security and potential algorithmic bias stemming from non-uniform detection performance.

References

Tejus Vignesh V (n.d). CS6461– Computer Vision Systems Lab-3: Real-Time Face Recognition: https://learn.ul.ie/content/enforced/69989-CS6461_SEM1_2025_6/Labs/Lab3-FacialRecognition/CS6461%20Lab3%20-%20Real-Time%20Face%20Recognition.pdf

Tejus Vignesh V (n.d). CS6461– Computer Vision Systems Lab-4: Image Signal Processing (ISP) vs Real-Time Face Recognition Performance: https://learn.ul.ie/d2l/common/dialogs/quickLink/quickLink.d2l?ou=69989&type=coursefile&fileId=Labs%2fLab4-ISP%2fCS6461+Lab4+-+ISP+vs+Real-Time+Face+Recognition+Performance.pdf

Tejus Vignesh V (n.d). CS6461– Computer Vision Systems Lab-5: Object Detection with TensorFlow Lite on JPG Images: https://learn.ul.ie/content/enforced/69989-CS6461_SEM1_2025_6/Labs/Lab5-TensorFlowJPEG/CS6461%20Lab5%20-%20Object%20Detection%20with%20TensorFlow%20Lite%20on%20JPG%20Images.pdf

A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Large-scale Video Classification with Deep Convolutional Neural Networks," *arXiv e-prints*, arXiv:1302.6379, Feb. 2013. [Online]. Available: https://arxiv.org/pdf/1302.6379

Shyam P., S. S. Sengar, K.-J. Yoon and K.-S. Kim, “Lightweight HDR Camera ISP for Robust Perception in Dynamic Illumination Conditions via Fourier Adversarial Networks,” arXiv preprint arXiv:2204.01795, 2022.

J. M. Llauradó, F. A. Pujol, D. Tomás, A. Visvizi, and M. Pujol, “Study of image sensors for enhanced face recognition at a distance in the Smart City context,” Scientific Reports, vol. 13, no. 14713, Sep. 2023, doi: 10.1038/s41598-023-40110-y.

Raspberry Pi Foundation, “Raspberry Pi 4 Model B Datasheet,” 2020. [Online]. Available: https://www.raspberrypi.com/documentation/

OKDO, “202-0456 5MP Camera User Manual,” Manuals+. [Online]. Available: https://manuals.plus/okdo/202-0456-5mp-camera-manual

OKdo 5MP Camera,” Debix. [Online]. Available: https://debix.io/hardware/5mp-camera.html

P. J. Reddie, “coco.names — object class names for YOLO / Darknet,” GitHub repository pjreddie/darknet.[Online]. Available: https://github.com/pjreddie/darknet/blob/master/data/coco.names

homohapiens, “yolov4-tiny.weights,” Hugging Face repository darknet-yolov4, accessed Nov. 29, 2025. [Online]. Available: https://huggingface.co/homohapiens/darknet-yolov4/blob/main/yolov4-tiny.weights

homohapiens, “yolov4-tiny.cfg,” Hugging Face repository darknet-yolov4, accessed Nov. 29, 2025. [Online]. Available: https://huggingface.co/homohapiens/darknet-yolov4/blob/main/yolov4-tiny.cfg